

## 3.2 URDF 基础

**所属阶段：** 第三阶段 · 坐标变换与机器人建模

**讲次数量：** 10 节

**预计时长：** ~120 分钟

**前置依赖：** 3.1 TF 坐标变换与数学基础；1.5 RViz 2 可视化工具入门

### 学习目标

完成本节后，学员将能够：

- 解释 URDF 的设计目的、核心元素（Link / Joint）及其在 ROS 生态中的角色
- 从零编写 URDF 文件，定义 Link 几何体并在 RViz 2 中可视化
- 使用 Material 标签为机器人部件添加颜色，用 Joint 连接多个 Link
- 区分 fixed / revolute / continuous / prismatic / floating / planar 六种关节类型
- 遵循编写 URDF 的标准流程，理解 URDF 与 TF 树的映射关系
- 从命令行独立启动 Robot State Publisher 和完整 URDF 可视化流程
- 了解真实 Mesh 模型（DAE/STL）在 URDF 中的引用方式

## 讲次 1：什么是 URDF

### 知识点

#### 机器人描述的需求

在 3.1 中，我们学习了 TF 系统如何管理坐标变换。但 TF 本身不回答一个更基础的问题：

**这个机器人长什么样？各部件尺寸多少？关节如何连接？**

URDF (Unified Robot Description Format, 统一机器人描述格式) 就是 ROS 生态中回答这个问题的标准格式。

#### URDF 的定义

URDF 是一种基于 XML 的机器人模型描述语言，用于定义：

| 元素                | 描述内容                       |
|-------------------|----------------------------|
| Link (连杆)         | 机器人的刚性部件：几何形状、质量、惯性、外观     |
| Joint (关节)        | 连接两个 Link 的运动关系：类型、位置、运动范围 |
| Material (材质)     | 可视化颜色与纹理                   |
| Transmission (传动) | 关节与执行器之间的传动关系 (Gazebo 仿真用) |
| Gazebo 扩展         | 仿真专用属性 (摩擦、传感器插件等)         |

#### URDF 在 ROS 2 workflow 中的位置

开发者编写

robot.urdf  
(XML 文件)

→

系统读取

robot\_description  
(参数字符串)

→

运行时效果

TF 树 + RViz 模型  
Gazebo 仿真模型  
MoveIt 运动规划

## URDF 文件的顶层结构

```
<?xml version="1.0"?>
<robot name="my_robot">

  <!-- 材质定义（可选，全局） -->
  <material name="blue">
    <color rgba="0 0 0.8 1"/>
  </material>

  <!-- 连杆定义 -->
  <link name="base_link">
    <visual>
      <geometry>
        <box size="0.4 0.3 0.1"/>
      </geometry>
    </visual>
  </link>

  <!-- 关节定义 -->
  <joint name="joint_name" type="fixed">
    <parent link="parent_link_name"/>
    <child link="child_link_name"/>
    <origin xyz="0 0 0" rpy="0 0 0"/>
  </joint>

</robot>
```

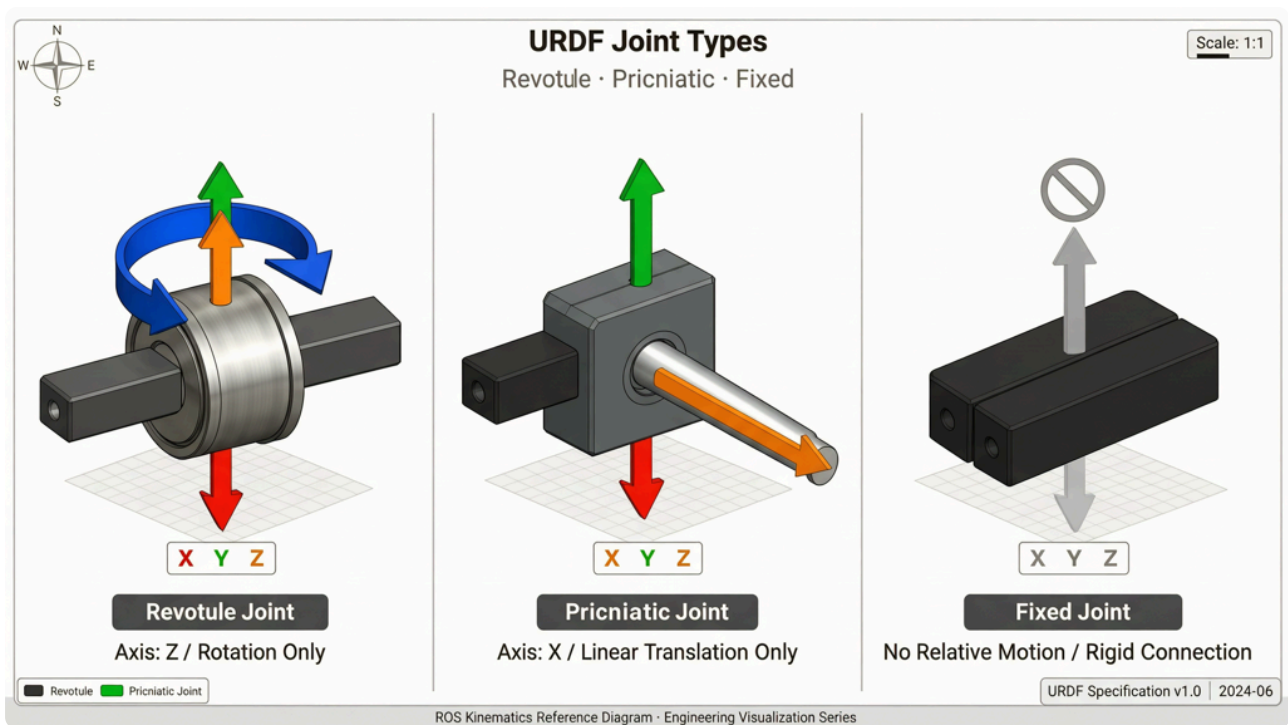
根元素 `<robot>` 的 `name` 属性是机器人名称，在整个 URDF 中唯一。

## Link 的子元素概览

```
<link name="...">
  <visual>      ← 可视化外观（RViz / Gazebo 渲染）
  <collision>   ← 碰撞几何体（运动规划 / 仿真碰撞检测）
  <inertial>    ← 质量与惯性（物理仿真必需）
</link>
```

| 子元素                    | 是否必需     | 用途                              |
|------------------------|----------|---------------------------------|
| <code>visual</code>    | 推荐       | RViz 中显示机器人外形                   |
| <code>collision</code> | 导航/仿真时需要 | 碰撞检测，通常比 <code>visual</code> 简化 |
| <code>inertial</code>  | 仿真时需要    | 质量、质心、惯性张量                      |

本讲先聚焦 `visual`，`collision` 和 `inertial` 将在 3.4 节深入。



## Joint 的子元素概览

```
<joint name="..." type="fixed|revolute|...">
  <parent link="..."/>      <!-- 父连杆（必需） -->
  <child link="..."/>      <!-- 子连杆（必需） -->
  <origin xyz="..." rpy="..."/> <!-- 关节位置（必需） -->
  <axis xyz="0 0 1"/>      <!-- 运动轴（旋转/移动关节必需） -->
  <limit lower="..." upper="..." effort="..." velocity="..."/> <!-- 运动范围 -->
</joint>
```

## URDF 的局限性

| 局限      | 说明               | 解决方案                     |
|---------|------------------|--------------------------|
| 不支持闭环机构 | 仅支持树形结构          | 使用 Gazebo 闭环插件或 xacro 技巧 |
| 不支持柔性体  | 所有 link 为刚体      | 外部有限元仿真                  |
| 文件冗长    | 大型机器人 URDF 可达数千行 | 使用 Xacro 宏（3.3 节）        |
| 仅静态描述   | 不含控制器、传感器驱动逻辑    | 配合 Launch 文件和驱动节点        |

查看系统自带的 URDF 示例

```
source /opt/ros/jazzy/setup.bash

# TurtleBot3 Burger URDF
cat /opt/ros/jazzy/share/turtlebot3_description/urdf/turtlebot3_burger.urdf | head -50

# 统计 link 和 joint 数量
grep -c '<link' /opt/ros/jazzy/share/turtlebot3_description/urdf/turtlebot3_burger.urdf
grep -c '<joint' /opt/ros/jazzy/share/turtlebot3_description/urdf/turtlebot3_burger.urdf
```

URDF 相关工具包

| 包 / 工具                | 作用                                  |
|-----------------------|-------------------------------------|
| urdfdom               | URDF 解析库                            |
| liburdfdom-tools      | check_urdf 、 urdf_to_graphviz 命令行工具 |
| robot_state_publisher | 将 URDF 转为 TF 变换                     |
| joint_state_publisher | 发布关节状态                              |

安装验证工具：

```
sudo apt install liburdfdom-tools
check_urdf --help
```

关键点

- URDF 是 ROS 的标准机器人模型描述格式，基于 XML，核心元素为 Link 和 Joint
  - Link 描述部件的几何与物理属性 (visual / collision / inertial)
  - Joint 描述两个 Link 之间的连接关系和运动类型
  - URDF 仅支持树形结构（无闭环），大型模型应使用 Xacro 简化
  - check\_urdf 是验证 URDF 语法正确性的第一步工具
-

## 讲次 2：创建第一个 URDF 文件并可视化 Link

### 知识点

#### 实验目标

创建一个最简单的 URDF 文件——仅包含一个 box 形状的 `base_link`，并在 RViz 2 中可视化。

#### 步骤 1：创建工作空间与功能包

```
# 创建工作空间
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws/src

# 创建描述功能包
ros2 pkg create --build-type ament_cmake my_robot_description
```

功能包目录结构：

```
my_robot_description/
├── CMakeLists.txt
├── package.xml
├── urdf/
│   └── my_robot.urdf ← 我们的 URDF 文件
└── ← 手动创建
```

```
mkdir -p ~/ros2_ws/src/my_robot_description/urdf
```

本仓库已提供可直接运行的配套源码：

```
stage3_code/my_robot_description
├── urdf/first_box.urdf      # 单个 base_link 方块
├── urdf/my_robot.urdf      # 彩色底盘 + 传感器支柱
├── urdf/diff_drive.urdf    # 差速两轮模型（含传感器支柱）
├── launch/display_xacro.launch.py # 可用 model:= 指定要显示的 URDF/Xacro 文件
├── launch/display.launch.py   # 固定加载最终版 my_robot.urdf.xacro（3.4/3.5 使用）
└── rviz/display.rviz
```

显示本节的纯 URDF 文件时，用 `display_xacro.launch.py model:=<URDF 绝对路径>`（`xacro` 命令可以原样展开纯 URDF）；`display.launch.py` 固定加载 3.4/3.5 的最终版 Xacro 模型，不适合本节的中间产物。也可以按本节后文用 `ros2 run robot_state_publisher ...` 手动启动。

如果使用配套源码，可复制到工作空间后构建：

```
mkdir -p ~/ros2_ai_stage3_ws/src
cp -r /path/to/ros2_ai/stage3_code/my_robot_description ~/ros2_ai_stage3_ws/src/
cd ~/ros2_ai_stage3_ws
source /opt/ros/jazzy/setup.bash
colcon build --packages-select my_robot_description
source install/setup.bash
```

## 步骤 2：编写第一个 URDF

创建 `~/ros2_ws/src/my_robot_description/urdf/my_robot.urdf`：

```
<?xml version="1.0"?>
<robot name="my_first_robot">

  <link name="base_link">
    <visual>
      <geometry>
        <box size="0.6 0.4 0.2"/>
      </geometry>
      <origin xyz="0 0 0.1" rpy="0 0 0"/>
    </visual>
  </link>

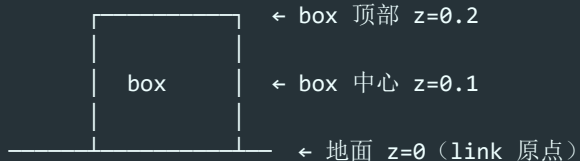
</robot>
```

逐行解读：

| 元素                                           | 含义                            |
|----------------------------------------------|-------------------------------|
| <code>&lt;box size="0.6 0.4 0.2"/&gt;</code> | 长 0.6m × 宽 0.4m × 高 0.2m 的长方体 |
| <code>&lt;origin xyz="0 0 0.1"&gt;</code>    | 视觉几何体中心在 link 原点上方 10cm       |
| 无 <code>&lt;joint&gt;</code>                 | 单 link 模型，无需关节                |

**为什么 origin z=0.1?** 让 box 底部贴合地面 (Z=0)，而非中心在原点：

侧视图：



## 步骤 3：安装 URDF 文件

修改 `CMakeLists.txt`，添加安装规则：

```
install(DIRECTORY urdf
  DESTINATION share/${PROJECT_NAME}
)
```

构建：

```
cd ~/ros2_ws
colcon build --packages-select my_robot_description
source install/setup.bash
```

#### 步骤 4：验证 URDF 语法

```
check_urdf ~/ros2_ws/install/my_robot_description/share/my_robot_description/urdf/my_robot.urdf
```

预期输出：

```
robot name is: my_first_robot
----- Successfully Parsed XML -----
root Link: base_link has 0 child(ren)
```

#### 步骤 5：启动可视化

终端 1——启动 `robot_state_publisher`：

```
source ~/ros2_ws/install/setup.bash
export URDF_PATH=$(ros2 pkg prefix my_robot_description)/share/my_robot_description/urdf/my_robot.urdf

ros2 run robot_state_publisher robot_state_publisher \
  --ros-args -p robot_description:="$(cat $URDF_PATH)"
```

终端 2——启动 RViz 2：

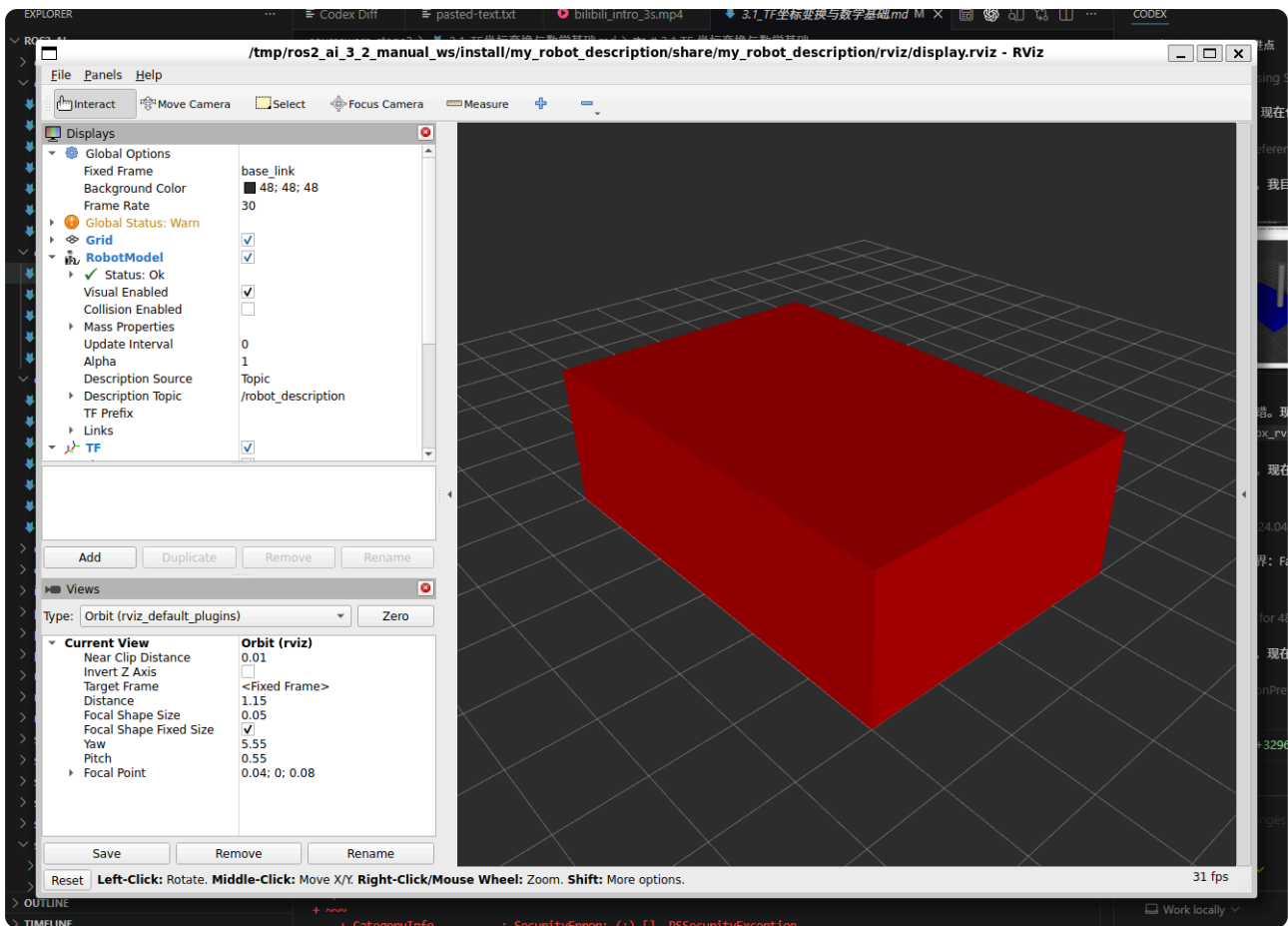
```
source ~/ros2_ws/install/setup.bash
ros2 run rviz2 rviz2
```

RViz 2 配置：

1. **Fixed Frame** → `base_link`
2. **Add** → **RobotModel**
3. **Description Topic** → `/robot_description`

此时应看到一个灰色长方体。





# 基本几何体类型

URDF 支持四种基本几何体：

| 类型  | 标签                            | 参数                                    | 典型用途        |
|-----|-------------------------------|---------------------------------------|-------------|
| 长方体 | <code>&lt;box&gt;</code>      | <code>size="长 宽 高"</code>             | 底盘、外壳       |
| 圆柱体 | <code>&lt;cylinder&gt;</code> | <code>radius="半径" length="长度"</code>  | 轮子、支柱       |
| 球体  | <code>&lt;sphere&gt;</code>   | <code>radius="半径"</code>              | 关节球头        |
| 网格  | <code>&lt;mesh&gt;</code>     | <code>filename="package://..."</code> | 复杂外形（讲次 10） |

示例——添加圆柱体轮子（预览，讲次 4 用 joint 连接）：

```

<link name="wheel">
  <visual>
    <geometry>
      <cylinder radius="0.05" length="0.04"/>
    </geometry>
    <origin xyz="0 0 0" rpy="1.5708 0 0"/>
    <!-- rpy 绕 X 转 90°，使圆柱轴沿 Y 方向（轮子惯例） -->
  </visual>
</link>

```

visual 元素的完整结构

```
<visual>
  <origin xyz="0 0 0" rpy="0 0 0"/>      <!-- 几何体相对于 link 原点的位姿 -->
  <geometry>                               <!-- 形状（必需） -->
    <box size="1 1 1"/>
  </geometry>
  <material name="blue">                  <!-- 颜色（可选） -->
    <color rgba="0 0 0.8 1"/>
  </material>
</visual>
```

常见问题

| 问题            | 原因                                        | 解决                  |
|---------------|-------------------------------------------|---------------------|
| RViz 中看不到模型   | Fixed Frame 错误或 robot_state_publisher 未运行 | 检查终端输出和 Fixed Frame |
| 模型悬浮/陷入地面     | visual origin z 值不正确                      | 调整 origin 使底部贴地     |
| check_urdf 报错 | XML 语法错误                                  | 检查标签闭合、属性引号         |
| 模型比例不对        | size 单位是米，不是厘米                            | 确认数值数量级             |

关键点

- 最简 URDF 只需一个 <robot> 根元素和一个 <link> 元素
  - <visual><geometry> 定义外观，支持 box / cylinder / sphere / mesh 四种基本形状
  - <origin> 定义几何体相对于 link 原点的位姿，注意 z 偏移使模型贴地
  - check\_urdf 验证语法，robot\_state\_publisher + RViz RobotModel 可视化
  - 尺寸单位统一为米（m），角度单位为弧度（rad）
-

## 讲次 3: URDF Material 标签——添加颜色

---

### 知识点

#### 为什么需要 Material

默认情况下，RViz 中所有几何体显示为同一灰色。Material 标签为机器人部件定义颜色，使模型更易辨识。

#### Material 的定义方式

URDF 支持两种定义 Material 的方式：

##### 方式 1: 全局定义 + 引用

```
<robot name="my_robot">
  <!-- 在 <robot> 下全局定义 -->
  <material name="blue">
    <color rgba="0.0 0.0 0.8 1.0"/>
  </material>
  <material name="red">
    <color rgba="0.8 0.0 0.0 1.0"/>
  </material>

  <link name="base_link">
    <visual>
      <geometry><box size="0.6 0.4 0.2"/></geometry>
      <material name="blue"/> <!-- 引用全局材质 -->
    </visual>
  </link>
</robot>
```

##### 方式 2: 内联定义

```
<link name="base_link">
  <visual>
    <geometry><box size="0.6 0.4 0.2"/></geometry>
    <material name="blue">
      <color rgba="0.0 0.0 0.8 1.0"/>
    </material>
  </visual>
</link>
```

#### rgba 颜色格式

```
rgba="R G B A"
```

| 分量 | 范围      | 含义              |
|----|---------|-----------------|
| R  | 0.0–1.0 | 红色分量            |
| G  | 0.0–1.0 | 绿色分量            |
| B  | 0.0–1.0 | 蓝色分量            |
| A  | 0.0–1.0 | 透明度 (1.0 = 不透明) |

常用颜色速查：

| 颜色 | rgba 值                       |
|----|------------------------------|
| 红色 | <code>0.8 0.0 0.0 1.0</code> |
| 绿色 | <code>0.0 0.8 0.0 1.0</code> |
| 蓝色 | <code>0.0 0.0 0.8 1.0</code> |
| 黄色 | <code>0.8 0.8 0.0 1.0</code> |
| 白色 | <code>1.0 1.0 1.0 1.0</code> |
| 黑色 | <code>0.1 0.1 0.1 1.0</code> |
| 橙色 | <code>1.0 0.5 0.0 1.0</code> |
| 灰色 | <code>0.5 0.5 0.5 1.0</code> |

**完整示例：带颜色的双部件模型**

更新 `my_robot.undf`：

```

<?xml version="1.0"?>
<robot name="my_first_robot">

  <material name="blue">
    <color rgba="0.0 0.0 0.8 1.0"/>
  </material>
  <material name="grey">
    <color rgba="0.5 0.5 0.5 1.0"/>
  </material>

  <link name="base_link">
    <visual>
      <geometry>
        <box size="0.6 0.4 0.2"/>
      </geometry>
      <origin xyz="0 0 0.1" rpy="0 0 0"/>
      <material name="blue"/>
    </visual>
  </link>

  <link name="sensor_mast">
    <visual>
      <geometry>
        <cylinder radius="0.03" length="0.3"/>
      </geometry>
      <origin xyz="0 0 0.15" rpy="0 0 0"/>
      <material name="grey"/>
    </visual>
  </link>

  <joint name="base_to_mast" type="fixed">
    <parent link="base_link"/>
    <child link="sensor_mast"/>
    <origin xyz="0 0 0.2" rpy="0 0 0"/>
  </joint>

</robot>

```

此模型包含： - 蓝色底盘 box ( $0.6 \times 0.4 \times 0.2$  m) - 灰色传感器支柱 cylinder (半径 3cm, 高 30cm) - 固定关节连接二者

## 验证与可视化

```

cd ~/ros2_ws
colcon build --packages-select my_robot_description
source install/setup.bash

check_urdf $(ros2 pkg prefix my_robot_description)/share/my_robot_description/urdf/my_robot.urdf

```

启动可视化（同讲次 2 流程），应看到蓝色底盘 + 灰色支柱。

## Material 与 Gazebo 材质的区别

| 维度   | URDF Material                               | Gazebo Material                              |
|------|---------------------------------------------|----------------------------------------------|
| 作用范围 | RViz 可视化                                    | Gazebo 仿真渲染                                  |
| 定义位置 | <code>&lt;visual&gt;&lt;material&gt;</code> | <code>&lt;gazebo reference="link"&gt;</code> |
| 颜色格式 | rgba 四元组                                    | Gazebo 特定标签                                  |
| 本讲范围 | ✓                                           | 3.4 节讲解                                      |

RViz 不读取 Gazebo 材质标签，反之亦然。若需两处颜色一致，需分别定义。

### 纹理贴图（了解）

Material 也支持纹理（本课程不深入）：

```
<material name="textured">
  <texture filename="package://my_robot_description/meshes/texture.png"/>
  <color rgba="1 1 1 1"/>
</material>
```

### 最佳实践

| 实践        | 原因                                                                      |
|-----------|-------------------------------------------------------------------------|
| 全局定义常用材质  | 避免重复，便于统一修改                                                             |
| 语义化命名     | <code>blue</code> 、 <code>chassis_grey</code> 比 <code>color1</code> 更清晰 |
| 功能部件用不同颜色 | 底盘、传感器、轮子用不同颜色便于辨识                                                      |
| 避免过多材质    | RViz 中过多颜色反而干扰观察                                                        |

### 关键点

- Material 通过 `rgba="R G B A"` 定义颜色，值域 0.0–1.0
  - 支持全局定义 + 引用和内联定义两种方式，推荐全局定义复用
  - 每个 link 的 visual 可独立指定材质，实现部件分色
  - URDF Material 仅影响 RViz；Gazebo 需单独配置材质
  - 颜色选择应服务于功能辨识，而非纯粹装饰
-



```

<?xml version="1.0"?>
<robot name="diff_drive_robot">

  <material name="blue">
    <color rgba="0.0 0.0 0.8 1.0"/>
  </material>
  <material name="black">
    <color rgba="0.1 0.1 0.1 1.0"/>
  </material>
  <material name="grey">
    <color rgba="0.5 0.5 0.5 1.0"/>
  </material>

  <!-- 底盘 -->
  <link name="base_link">
    <visual>
      <geometry><box size="0.5 0.3 0.1"/></geometry>
      <origin xyz="0 0 0.05" rpy="0 0 0"/>
      <material name="blue"/>
    </visual>
  </link>

  <!-- 左轮 -->
  <link name="left_wheel">
    <visual>
      <geometry><cylinder radius="0.05" length="0.04"/></geometry>
      <origin xyz="0 0 0" rpy="1.5708 0 0"/>
      <material name="black"/>
    </visual>
  </link>

  <!-- 右轮 -->
  <link name="right_wheel">
    <visual>
      <geometry><cylinder radius="0.05" length="0.04"/></geometry>
      <origin xyz="0 0 0" rpy="1.5708 0 0"/>
      <material name="black"/>
    </visual>
  </link>

  <!-- 左轮关节 -->
  <joint name="left_wheel_joint" type="continuous">
    <parent link="base_link"/>
    <child link="left_wheel"/>
    <origin xyz="0 0.175 0.05" rpy="0 0 0"/>
    <axis xyz="0 1 0"/>
  </joint>

  <!-- 右轮关节 -->
  <joint name="right_wheel_joint" type="continuous">
    <parent link="base_link"/>
    <child link="right_wheel"/>
    <origin xyz="0 -0.175 0.05" rpy="0 0 0"/>
    <axis xyz="0 1 0"/>
  </joint>

  <!-- 传感器支柱（复习 fixed joint：一个 link 可以有多个子节点） -->

```



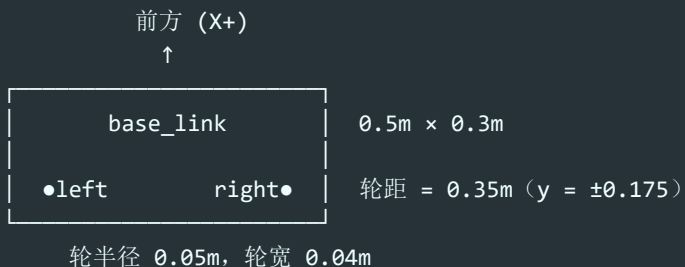
```
<link name="sensor_mast">
  <visual>
    <origin xyz="0 0 0.15" rpy="0 0 0"/>
    <geometry><cylinder radius="0.025" length="0.3"/></geometry>
    <material name="grey"/>
  </visual>
</link>

<joint name="base_to_mast" type="fixed">
  <parent link="base_link"/>
  <child link="sensor_mast"/>
  <origin xyz="0.15 0 0.1" rpy="0 0 0"/>
</joint>

</robot>
```

## 尺寸计算说明

俯视图:



| 参数          | 值          | 计算                            |
|-------------|------------|-------------------------------|
| 底盘高度        | 0.1m       | box size Z                    |
| 底盘 origin z | 0.05m      | 使底部贴地                         |
| 轮子 origin x | 0          | 前后居中                          |
| 轮子 origin y | ±0.175     | 轮距 0.35m 的一半 (0.35/2 = 0.175) |
| 轮子 origin z | 0.05       | 轮心与底盘中心同高                     |
| 圆柱 rpy      | 1.5708 0 0 | 绕 X 转 90°, 轴沿 Y               |

## 验证 URDF 树结构

```
check_urdf ~/ros2_ws/install/my_robot_description/share/my_robot_description/urdf/diff_drive.urdf

# 生成结构图（输出文件名取自 robot name, 即 diff_drive_robot.pdf/.gv）
urdf_to_graphviz diff_drive.urdf
evince diff_drive_robot.pdf
```

预期树结构:

```
base_link
├─ left_wheel
├─ right_wheel
└─ sensor_mast
```

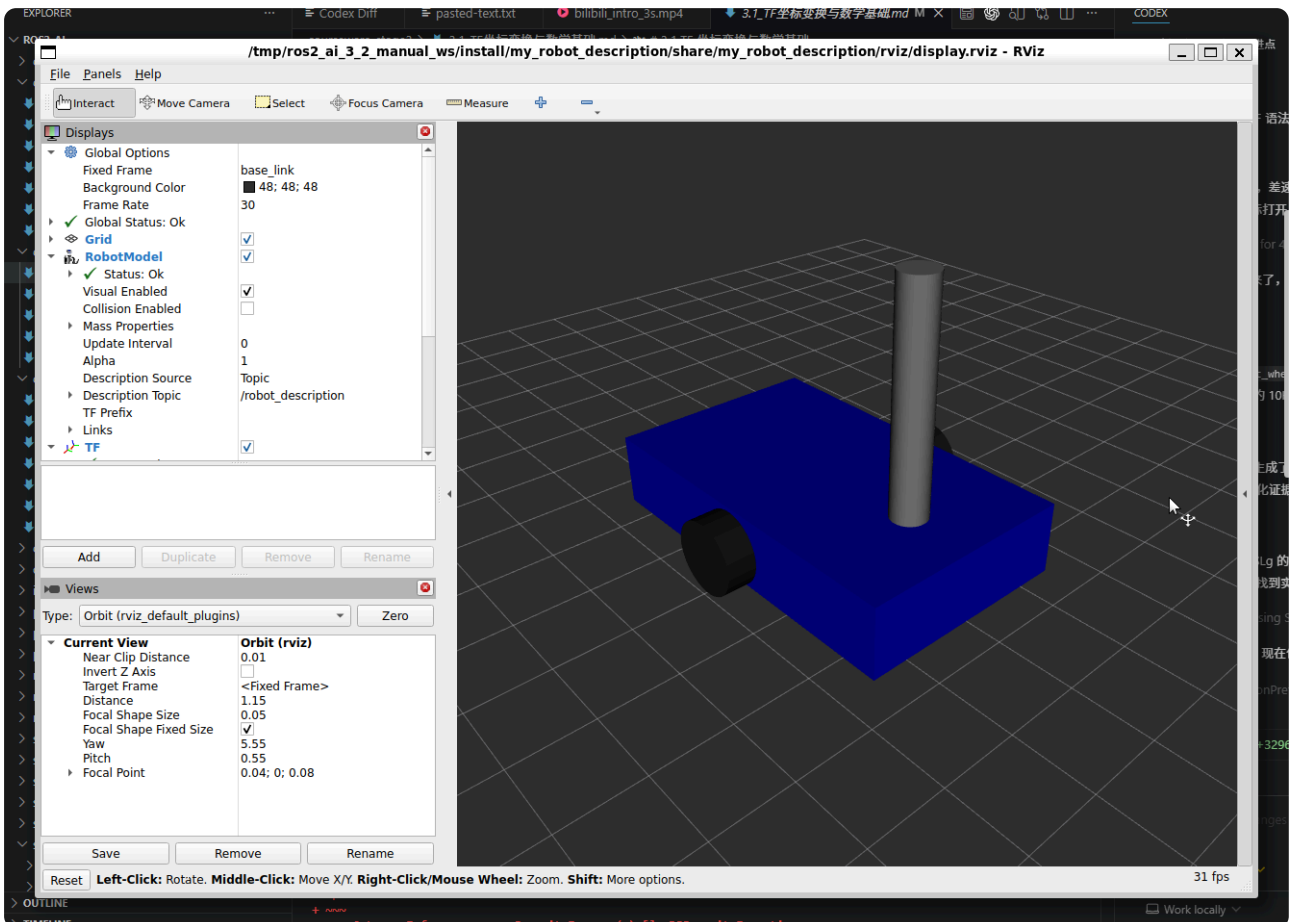
## 可视化并驱动关节

```
# 终端 1
source ~/ros2_ws/install/setup.bash
export URDF_PATH=$(ros2 pkg prefix my_robot_description)/share/my_robot_description/urdf/diff_dr
ros2 run robot_state_publisher robot_state_publisher \
  --ros-args -p robot_description:="$(cat $URDF_PATH)"

# 终端 2
ros2 run joint_state_publisher_gui joint_state_publisher_gui

# 终端 3
ros2 run rviz2 rviz2
```

在 GUI 中拖动 `left_wheel_joint` 和 `right_wheel_joint` 滑块，观察轮子旋转。



## origin 的坐标系约定

Joint 的 `<origin>` 定义关节坐标系在 parent link 中的位姿；当关节位置为 0 时，child link 坐标系与该关节坐标系重合，因此可理解为 **child link 的零位位姿**：

```
T_parent_child = origin(xyz, rpy)
```

含义: `child` 的原点在 `parent` 中的位置是 `xyz`,  
`child` 的朝向相对 `parent` 旋转 `rpy`

常见错误

| 错误                         | 后果        | 修正                                        |
|----------------------------|-----------|-------------------------------------------|
| parent 和 child 写反          | TF 树方向颠倒  | 确认 parent 是父级 link                        |
| origin y 方向搞反              | 左轮跑到右边    | y 正方向为左                                   |
| 忘记 axis 标签                 | 旋转关节无法运动  | 添加 <code>&lt;axis xyz="0 1 0"/&gt;</code> |
| 同一 link 作为两个 joint 的 child | URDF 解析失败 | 每个 link 只能有一个父 joint                      |

关键点

- Joint 通过 parent/child 连接两个 Link, origin 定义相对位姿
  - Joint 的父子关系直接映射为 TF 树的父子关系
  - 每个 link 只能是一个 joint 的 child, 保证 URDF 为树形结构
  - 轮子的 visual origin 常需 rpy 旋转使圆柱轴沿正确方向
  - `urdf_to_graphviz` 生成树形结构图, 是检查连接关系的直观工具
-

# 讲次 5：URDF 中不同类型的 Joint

## 知识点

### 六种关节类型

URDF 定义了 6 种基础关节模型：

| 类型         | 运动方式              | 自由度    | 典型应用       |
|------------|-------------------|--------|------------|
| fixed      | 不可动               | 0      | 传感器安装、刚性连接 |
| continuous | 无限旋转              | 1 (旋转) | 轮子、风扇      |
| revolute   | 有限角度旋转            | 1 (旋转) | 机械臂关节、铰链   |
| prismatic  | 线性滑动              | 1 (平移) | 伸缩杆、升降台    |
| floating   | 六自由度              | 6      | 无人机、自由漂浮物  |
| planar     | 平面内两个平移 + 绕平面法向旋转 | 3      | 较少使用       |

| Joint 类型   | 运动方式     | 示意图          |
|------------|----------|--------------|
| fixed      | 无运动      | ■—■ (刚性连接)   |
| revolute   | 有限角度旋转   | ■—○—■ (带限位)  |
| continuous | 无限旋转     | ■—∞—■ (如车轮)  |
| prismatic  | 直线滑动     | ■—↔—■ (伸缩)   |
| floating   | 6自由度自由运动 | ■ ... ■ (浮动) |
| planar     | 平面内运动    | ■—↖—■ (XY平面) |

### fixed — 固定关节

两个 link 之间无相对运动：

```
<joint name="laser_mount" type="fixed">
  <parent link="base_link"/>
  <child link="laser_link"/>
  <origin xyz="0.2 0 0.15" rpy="0 0 0"/>
</joint>
```

- 不需要 `<axis>` 和 `<limit>`
- 变换发布到 `/tf_static`
- 最常用的关节类型（传感器、支架）

## continuous — 连续旋转关节

可无限旋转，无角度限制：

```
<joint name="wheel_joint" type="continuous">
  <parent link="base_link"/>
  <child link="wheel_link"/>
  <origin xyz="0 0.15 0.05" rpy="0 0 0"/>
  <axis xyz="0 1 0"/>
</joint>
```

- 建议显式指定 `<axis>`（旋转轴方向）；省略时 URDF 默认为 `1 0 0`
- 不需要 `lower/upper` 角度上下限；接入控制器时仍应根据硬件能力配置速度和力矩约束
- 变换发布到 `/tf`（随 `joint_states` 更新）

## revolute — 旋转关节

有限角度范围的旋转关节：

```
<joint name="arm_joint" type="revolute">
  <parent link="base_link"/>
  <child link="arm_link"/>
  <origin xyz="0 0 0.1" rpy="0 0 0"/>
  <axis xyz="0 0 1"/>
  <limit lower="-1.57" upper="1.57" effort="10.0" velocity="1.0"/>
</joint>
```

| limit 属性 | 含义   | 单位    |
|----------|------|-------|
| lower    | 最小角度 | 弧度    |
| upper    | 最大角度 | 弧度    |
| effort   | 最大力矩 | N·m   |
| velocity | 最大速度 | rad/s |

## prismatic — 滑动关节

沿轴线线性移动：

```
<joint name="lift_joint" type="prismatic">
  <parent link="base_link"/>
  <child link="platform_link"/>
  <origin xyz="0 0 0.2" rpy="0 0 0"/>
  <axis xyz="0 0 1"/>
  <limit lower="0" upper="0.5" effort="100.0" velocity="0.1"/>
</joint>
```

| limit 属性 | 含义   | 单位  |
|----------|------|-----|
| lower    | 最小位移 | 米   |
| upper    | 最大位移 | 米   |
| effort   | 最大力  | N   |
| velocity | 最大速度 | m/s |

### floating — 浮动关节

六自由度运动（3 平移 + 3 旋转）：

```
<!-- floating 关节的 parent 必须是已定义的 link -->
<link name="world"/>

<joint name="world_to_base" type="floating">
  <parent link="world"/>
  <child link="base_link"/>
</joint>
```

- 通常用于将机器人连接到虚拟 world link
- 下游工具对 floating/planar 的支持并不完全一致；移动机器人通常由里程计或定位节点发布 odom -> base\_link 等 TF，而不在 URDF 中使用 floating joint

### planar — 平面关节

planar joint 允许 child 在与 <axis> 法向垂直的平面内做两个方向的平移，并绕该法向旋转，合计 3 个自由度。它在常见移动机器人 ROS 2 建模链中较少直接使用，平面位姿通常交给里程计和定位系统。

### axis 标签详解

<axis xyz="ax ay az"/> 定义关节运动轴方向（在 joint 坐标系中）。对 revolute、continuous 和 prismatic，该标签在语法上可省略，默认轴是 1 0 0；工程中应显式写出，避免模型朝向变更后出现隐蔽错误：

| 场景          | axis 值 | 含义     |
|-------------|--------|--------|
| 轮子（绕 Y 轴转）  | 0 1 0  | 常见差速轮  |
| 机械臂（绕 Z 轴转） | 0 0 1  | 常见竖直关节 |
| 升降台（沿 Z 轴移） | 0 0 1  | 竖直滑动   |

axis 应使用非零向量，并建议显式写成单位向量。

## 关节类型对 TF 的影响

```
fixed joint:
  → 发布到 /tf_static
  → 变换 = origin(xyz, rpy), 永不改变

continuous / revolute:
  → 发布到 /tf
  → 变换 = origin × Rot_axis(joint_position)
  → 需要 /joint_states 驱动

prismatic:
  → 发布到 /tf
  → 变换 = origin × Trans_axis(joint_position)
  → 需要 /joint_states 驱动
```

## 完整示例：机械臂片段

```
<!-- 3-DOF 机械臂 -->
<joint name="joint1" type="revolute">
  <parent link="base_link"/>
  <child link="link1"/>
  <origin xyz="0 0 0.1" rpy="0 0 0"/>
  <axis xyz="0 0 1"/>
  <limit lower="-3.14" upper="3.14" effort="50" velocity="2.0"/>
</joint>

<joint name="joint2" type="revolute">
  <parent link="link1"/>
  <child link="link2"/>
  <origin xyz="0 0 0.3" rpy="0 0 0"/>
  <axis xyz="0 1 0"/>
  <limit lower="-1.57" upper="1.57" effort="30" velocity="1.5"/>
</joint>

<joint name="joint3" type="revolute">
  <parent link="link2"/>
  <child link="link3"/>
  <origin xyz="0 0 0.25" rpy="0 0 0"/>
  <axis xyz="0 1 0"/>
  <limit lower="-1.57" upper="1.57" effort="20" velocity="1.0"/>
</joint>
```

## 选择关节类型的决策树

```
部件之间有相对运动吗？
├─ 否 → fixed
└─ 是 → 运动类型？
    ├── 旋转 → 角度有限制吗？
    │   ├── 是 → revolute
    │   └─ 否 → continuous
    ├── 平移 → prismatic
    └─ 全自由度 → floating
```

## 关键点

- URDF 有 6 种关节类型：fixed（最常用）、continuous、revolute、prismatic、floating、planar
  - fixed 关节发布到 `/tf_static`；运动关节发布到 `/tf` 并需要 `/joint_states`
  - 运动关节建议显式指定 `<axis>`；revolute 和 prismatic 必须提供完整的 `<limit>`
  - continuous 无角度限制，revolute 有 lower/upper 角度限制
  - 选择关节类型的关键是分析部件间的实际运动方式
-

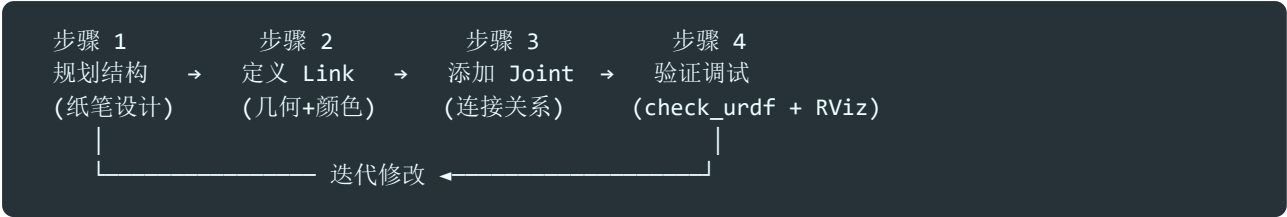


# 讲次 6：编写 URDF 的标准流程

## 知识点

### 标准流程总览

编写 URDF 遵循"自底向上、逐步验证"的流程：



### 步骤 1：规划机器人结构

在写代码之前，先用纸笔画出机器人结构：

规划内容：

1. 有哪些部件？（每个部件 = 一个 link）
2. 部件如何连接？（每个连接 = 一个 joint）
3. 关键尺寸？（长宽高、轮距、传感器位置）
4. 关节类型？（fixed / revolute / continuous ...）

示例——三轮全向机器人规划：



| Link         | 形状       | 尺寸                |
|--------------|----------|-------------------|
| base_link    | box      | 0.4 × 0.3 × 0.1 m |
| left_wheel   | cylinder | r=0.05, l=0.04 m  |
| right_wheel  | cylinder | r=0.05, l=0.04 m  |
| front_caster | sphere   | r=0.02 m          |

| Joint             | Type       | Parent → Child           |
|-------------------|------------|--------------------------|
| left_wheel_joint  | continuous | base_link → left_wheel   |
| right_wheel_joint | continuous | base_link → right_wheel  |
| caster_joint      | fixed      | base_link → front_caster |

## 步骤 2: 定义 Link (从根 link 开始)

**原则:** 先定义父 link, 再定义子 link。

```
<!-- 顺序: 先 base_link (根), 再子部件 -->
<link name="base_link">...</link>
<link name="left_wheel">...</link>
<link name="right_wheel">...</link>
<link name="front_caster">...</link>
```

每添加一个 link 后立即验证:

```
check_urdf my_robot.urdf
# 孤立 link 会显示为 root link
```

## 步骤 3: 添加 Joint (连接 link)

**原则:** 先添加靠近根节点的 joint, 再向外扩展。

```
<!-- 先连接直接挂在 base_link 上的关节 -->
<joint name="left_wheel_joint" type="continuous">...</joint>
<joint name="right_wheel_joint" type="continuous">...</joint>
<joint name="caster_joint" type="fixed">...</joint>
```

每添加一个 joint 后验证 TF 树:

```
check_urdf my_robot.urdf
urdf_to_graphviz my_robot.urdf
```

## 步骤 4: 验证与调试

完整的验证清单:

| 检查项    | 工具                                     | 预期结果                |
|--------|----------------------------------------|---------------------|
| XML 语法 | <code>check_urdf</code>                | Successfully Parsed |
| 树结构    | <code>urdf_to_graphviz</code>          | 无孤立节点               |
| 可视化    | RViz RobotModel                        | 模型外观正确              |
| TF 树   | <code>view_frames</code>               | 与 URDF 结构一致         |
| 关节运动   | <code>joint_state_publisher_gui</code> | 运动关节可驱动             |
| 尺寸合理性  | 目视 + 尺子 (RViz Grid)                    | 比例协调                |

### 迭代开发技巧

#### 技巧 1：增量构建

```
第 1 轮: base_link only → 验证
第 2 轮: + 两个轮子 → 验证
第 3 轮: + 传感器支架 → 验证
第 4 轮: + 传感器 → 验证
```

#### 技巧 2：使用 RViz Grid 作为尺寸参考

```
RViz → Add → Grid
Cell Size = 0.1 (每格 10cm)
可直观判断模型尺寸是否合理
```

#### 技巧 3：先固定关节，后改为运动关节

开发时先将所有 joint 设为 `fixed`，确认位置正确后再改为 `continuous` 或 `revolute`。

### 命名规范

| 元素            | 命名规则                           | 示例                                               |
|---------------|--------------------------------|--------------------------------------------------|
| robot name    | 小写 + 下划线                       | <code>diff_drive_robot</code>                    |
| link name     | 小写 + 下划线, 语义化                  | <code>base_link</code> , <code>left_wheel</code> |
| joint name    | link名 + <code>_joint</code> 后缀 | <code>left_wheel_joint</code>                    |
| material name | 颜色或用途                          | <code>blue</code> , <code>chassis_grey</code>    |

ROS 约定：- 根 link 通常命名为 `base_link` - 传感器 link 常用 `_link` 后缀： `laser_link`, `camera_link` - 地面投影： `base_footprint`

## 文件组织建议

```
my_robot_description/
├── urdf/
│   ├── my_robot.urdf          # 主 URDF 文件
│   └── my_robot_gazebo.urdf    # Gazebo 扩展（后续）
├── meshes/                    # 3D 模型文件
│   ├── chassis.dae
│   └── wheel.stl
├── launch/
│   └── display.launch.py      # 可视化启动文件
├── rviz/
│   └── display.rviz           # RViz 配置
├── CMakeLists.txt
└── package.xml
```

## 常用调试命令汇总

```
# 语法检查
check_urdf robot.urdf

# 结构图
urdf_to_graphviz robot.urdf

# 启动可视化
ros2 run robot_state_publisher robot_state_publisher \
  --ros-args -p robot_description:="$(cat robot.urdf)"
ros2 run joint_state_publisher_gui joint_state_publisher_gui
ros2 run rviz2 rviz2

# TF 验证
ros2 run tf2_tools view_frames
ros2 run tf2_ros tf2_echo base_link wheel_link
```

## 关键点

- 编写 URDF 遵循"规划 → 定义 Link → 添加 Joint → 验证"四步流程
  - 增量构建：每添加一个部件立即验证，避免错误累积
  - 先 fixed 后 motion：开发时先用 fixed 确认位置，再改为运动关节
  - 命名规范：小写 + 下划线, joint 名 = link 名 + `_joint`
  - `check_urdf` + `urdf_to_graphviz` + RViz 是验证三件套
-

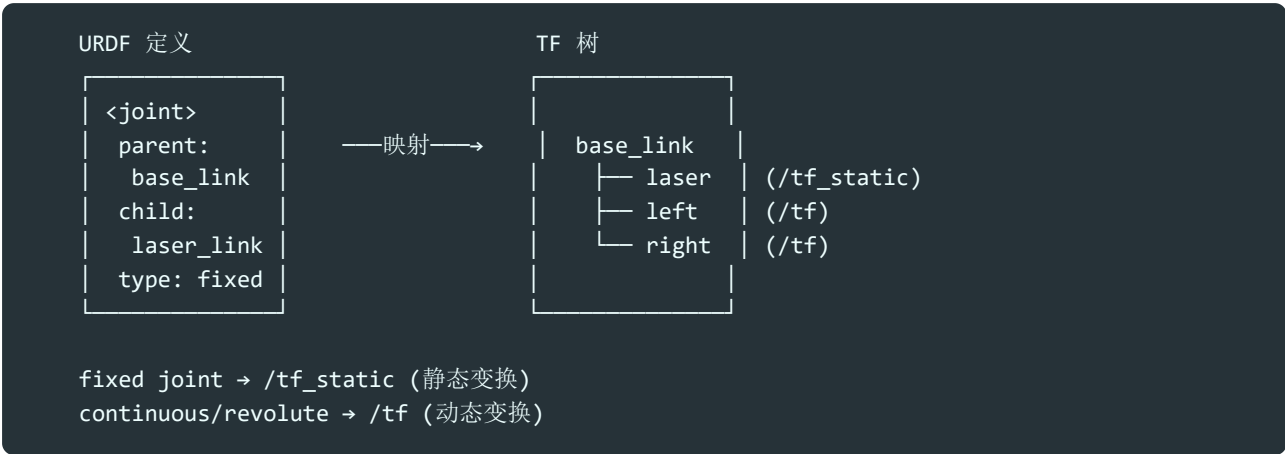
# 讲次 7: URDF 与 TF 树的对应关系

## 知识点

### 核心映射规则

URDF 与 TF 树之间存在一一对应的映射关系:

| URDF 元素                                         | TF 树元素                                                  |
|-------------------------------------------------|---------------------------------------------------------|
| <code>&lt;link name="A"&gt;</code>              | <code>frame: A</code>                                   |
| <code>&lt;joint parent="A" child="B"&gt;</code> | <code>变换: A → B</code><br><code>(parent → child)</code> |
| <code>&lt;origin xyz rpy&gt;</code>             | 变换的平移 + 旋转                                              |
| <code>joint position</code>                     | 动态变换增量                                                  |



### 完整映射示例

以下 URDF:

```

<robot name="my_robot">
  <link name="base_link"/>
  <link name="laser_link"/>
  <link name="wheel_link"/>

  <joint name="laser_joint" type="fixed">
    <parent link="base_link"/>
    <child link="laser_link"/>
    <origin xyz="0.2 0 0.15" rpy="0 0 0"/>
  </joint>

  <joint name="wheel_joint" type="continuous">
    <parent link="base_link"/>
    <child link="wheel_link"/>
    <origin xyz="0 0.15 0.05" rpy="0 0 0"/>
    <axis xyz="0 1 0"/>
  </joint>
</robot>

```

对应 TF 树:

```

base_link (root)
├── laser_link    [fixed: xyz=(0.2, 0, 0.15), 发布到 /tf_static]
└── wheel_link    [continuous: xyz=(0, 0.15, 0.05), 发布到 /tf]

```

## fixed joint 的 TF 行为

URDF:

```

<joint type="fixed">
  <origin xyz="0.2 0 0.15" rpy="0 0 0.1"/>

```

TF 行为:

1. 启动时计算一次变换
2. 发布到 `/tf_static`
3. 永不更新

查询:

```

tf2_echo base_link laser_link
→ Translation: [0.2, 0.0, 0.15]
→ RPY: [0.0, 0.0, 0.1]

```

## continuous joint 的 TF 行为

URDF:

```
<joint type="continuous">
  <origin xyz="0 0.15 0.05" rpy="0 0 0"/>
  <axis xyz="0 1 0"/>
```

TF 行为:

1. 收到 `/joint_states` 中 `wheel_joint` 的位置  $\theta$
2. 计算:  $T = \text{origin} \times \text{Rot}_y(\theta)$
3. 发布到 `/tf`

当  $\theta = 1.57$  ( $90^\circ$ ) 时:

```
tf2_echo base_link wheel_link
→ 轮子坐标系相对 base_link 旋转了  $90^\circ$ 
```

## 验证对应关系

启动 `diff_drive.urdf` 模型:

```
source ~/ros2_ws/install/setup.bash
export URDF_PATH=$(ros2 pkg prefix my_robot_description)/share/my_robot_description/urdf/diff_drive.urdf

# 终端 1
ros2 run robot_state_publisher robot_state_publisher \
  --ros-args -p robot_description:="$(cat $URDF_PATH)"

# 终端 2
ros2 run joint_state_publisher_gui joint_state_publisher_gui
```

对比 URDF 与 TF 输出:

```
# 查看 URDF 中左轮 joint 的 origin
grep -A5 'left_wheel_joint' $URDF_PATH

# 查看 TF 输出 (joint position = 0 时)
ros2 run tf2_ros tf2_echo base_link left_wheel
```

两者应完全一致。

## URDF 中没有对应 TF 的情况

以下情况 URDF 不直接产生 TF:

| 情况                | 说明        | TF 来源           |
|-------------------|-----------|-----------------|
| map → odom        | 定位系统变换    | AMCL / SLAM 节点  |
| odom → base_link  | 里程计变换     | 底盘驱动节点          |
| world → base_link | 仿真世界变换    | 仿真插件/桥接节点（按需配置） |
| 无 joint 的孤立 link  | URDF 语法错误 | 不应出现            |

robot\_state\_publisher 的解析流程

```
1. 读取 URDF XML
2. 解析所有 <link> → 注册 frame 名
3. 解析所有 <joint> → 建立父子关系
4. 对 fixed joint → 计算变换 → 发布 /tf_static
5. 订阅 /joint_states → 对运动 joint 计算变换 → 发布 /tf
6. 从本节点的 robot_description 参数读取模型，并发布 /robot_description 话题
```

常见不一致问题

| 现象           | URDF 问题                    | 排查方法                      |
|--------------|----------------------------|---------------------------|
| TF 树缺少 frame | link 未通过 joint 连接          | check_urdf 查看孤立 link      |
| 位置偏移         | origin xyz 值错误             | 对比 URDF origin 与 tf2_echo |
| 朝向错误         | origin rpy 值错误             | 检查 rpy 角度（弧度 vs 度）        |
| 关节不转         | joint type 或 axis 错误       | 确认 type 和 axis 设置         |
| 两个 parent    | 一个 link 被两个 joint 作为 child | check_urdf 报错             |

对照表：URDF 元素 → TF 查询

| 你想知道什么   | 查 URDF                    | 查 TF                          |
|----------|---------------------------|-------------------------------|
| 有哪些坐标系   | 所有 <link name>            | view_frames                   |
| 坐标系如何连接  | 所有 <joint> 的 parent/child | frames_*.gv / view_frames PDF |
| 固定部件位置   | <origin xyz rpy>          | tf2_echo parent child         |
| 运动部件当前位置 | joint type + axis         | tf2_echo （实时）                 |
| 完整树结构    | urdf_to_graphviz          | view_frames PDF               |

关键点

- URDF 的每个 link 对应 TF 树的一个 frame，每个 joint 对应一条变换边



- fixed joint → `/tf_static` (不变) ; 运动 joint → `/tf` (随 joint\_states 更新)
  - joint 的 origin 定义了 child 在 parent 中的固定位姿, 关节运动在此基础上叠加
  - `tf2_echo` 的输出应与 URDF origin 一致 (joint position = 0 时)
  - map/odom 等导航坐标系不在 URDF 中, 由其他节点发布
-

## 讲次 8：从命令行启动 Robot State Publisher

### 知识点

#### 本讲目标

不依赖 Launch 文件，纯粹用命令行启动 `robot_state_publisher`，理解其参数、输入和输出。

#### 最小启动命令

```
source /opt/ros/jazzy/setup.bash

# 方式 1: 直接传入 URDF 文件内容
ros2 run robot_state_publisher robot_state_publisher \
  --ros-args -p robot_description:="$(cat /path/to/robot.urdf)"
```

```
# 方式 2: 使用已有参数（若之前设置过）
ros2 run robot_state_publisher robot_state_publisher \
  --ros-args -p robot_description:="$(xacro /path/to/robot.xacro)"
```

#### 关键参数

| 参数                             | 类型     | 默认值   | 说明                    |
|--------------------------------|--------|-------|-----------------------|
| <code>robot_description</code> | string | 无（必需） | URDF XML 字符串          |
| <code>publish_frequency</code> | double | 50.0  | TF 发布频率 (Hz)          |
| <code>ignore_timestamp</code>  | bool   | false | 是否忽略 joint_states 时间戳 |
| <code>frame_prefix</code>      | string | ""    | 所有 frame 名前缀          |

**Jazzy 变更：** `use_tf_static` 参数已移除。固定关节变换始终发布到 `/tf_static`，并使用 `transient_local` QoS，订阅者无需额外配置。

查看所有参数应在节点启动后通过 `ros2 param` 完成：

```
ros2 param list /robot_state_publisher
ros2 param get /robot_state_publisher publish_frequency
ros2 param get /robot_state_publisher frame_prefix
```

## 验证节点运行状态

```
# 检查节点是否启动
ros2 node list
# 输出应包含 /robot_state_publisher

# 查看节点详情
ros2 node info /robot_state_publisher

# 查看发布的参数
ros2 param list /robot_state_publisher
ros2 param get /robot_state_publisher robot_description
ros2 param get /robot_state_publisher publish_frequency
```

## 检查输出话题

```
# robot_state_publisher 发布的话题
ros2 node info /robot_state_publisher

# 预期输出:
# Publishers:
#   /tf: tf2_msgs/msg/TFMessage
#   /tf_static: tf2_msgs/msg/TFMessage
#   /robot_description: std_msgs/msg/String
# Subscribers:
#   /joint_states: sensor_msgs/msg/JointState
```

```
# 查看 /tf_static 内容
ros2 topic echo /tf_static --once

# 查看 /robot_description 是否发布
ros2 topic echo /robot_description --once
```

## 配合 joint\_state\_publisher 启动

运动关节需要 `/joint_states` 输入:

```
# 终端 1: robot_state_publisher
source ~/ros2_ws/install/setup.bash
URDF=$(ros2 pkg prefix my_robot_description)/share/my_robot_description/urdf/diff_drive.urdf
ros2 run robot_state_publisher robot_state_publisher \
  --ros-args -p robot_description:="$(cat $URDF)"

# 终端 2: 关节状态发布 (无 GUI 版)
ros2 run joint_state_publisher joint_state_publisher

# 终端 3: 验证
ros2 topic hz /joint_states
ros2 topic hz /tf
ros2 run tf2_ros tf2_echo base_link left_wheel
```

## 使用 frame\_prefix

多机器人场景下，用 `frame_prefix` 避免 frame 名冲突：

```
# 机器人 1
ros2 run robot_state_publisher robot_state_publisher \
  --ros-args \
  -p robot_description:="$(cat robot.urdf)" \
  -p frame_prefix:"robot1/"

# 机器人 2
ros2 run robot_state_publisher robot_state_publisher \
  --ros-args \
  -p robot_description:="$(cat robot.urdf)" \
  -p frame_prefix:"robot2/"
```

此时 TF 树中 frame 名为 `robot1/base_link`、`robot2/base_link` 等。

## 调试技巧

查看 URDF 是否正确加载：

```
# 如果 robot_description 为空，说明文件读取失败
ros2 param get /robot_state_publisher robot_description | head -5
```

查看哪些 joint 在等待 joint\_states：

```
# 启动时不发布 /joint_states，观察日志
ros2 run robot_state_publisher robot_state_publisher \
  --ros-args -p robot_description:="$(cat diff_drive.urdf)" \
  --ros-args --log-level robot_state_publisher:=debug
```

手动发布 joint\_states 测试：

```
ros2 topic pub /joint_states sensor_msgs/msg/JointState \
  "{header: {stamp: {sec: 0, nanosec: 0}, frame_id: ''}, \
  name: ['left_wheel_joint', 'right_wheel_joint'], \
  position: [1.57, 0.0], \
  velocity: [], effort: []}" --once
```

与 Launch 文件的对比

| 维度    | 命令行        | Launch 文件 |
|-------|------------|-----------|
| 适用场景  | 学习、调试、快速验证 | 生产部署、复杂启动 |
| 参数管理  | 手动传入       | 声明式配置     |
| 多节点启动 | 需多个终端      | 一个命令全部启动  |
| 可重复性  | 低（依赖手动操作）  | 高（版本化管理）  |

命令行是理解原理的最佳方式；Launch 是工程实践的标准方式（3.4 节将创建 Launch 文件）。

关键点

- `robot_state_publisher` 的最小启动只需 `robot_description` 参数
  - 节点发布 `/tf`、`/tf_static` 和 `/robot_description` 三个话题
  - 运动关节需要 `/joint_states` 输入，否则运动 link 的 TF 不更新
  - `frame_prefix` 用于多机器人场景的 frame 名隔离
  - 命令行启动是理解原理的基础，Launch 是工程化的下一步
-

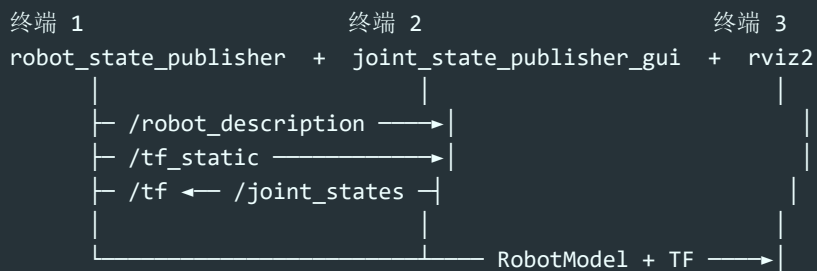
## 讲次 9：从终端启动完整 URDF 可视化流程

### 知识点

#### 本讲目标

整合前序知识，用一组终端命令完成从 URDF 文件到 RViz 2 可视化的完整流程，并保存可复现的脚本。

#### 完整流程架构



#### 步骤 0：准备 URDF 文件

使用讲次 4 创建的 `diff_drive.urdf`，确认已构建：

```
cd ~/ros2_ws
colcon build --packages-select my_robot_description
source install/setup.bash

# 验证
export URDF_FILE=$(ros2 pkg prefix my_robot_description)/share/my_robot_description/urdf/diff_dr
check_urdf $URDF_FILE
```

#### 步骤 1：启动 `robot_state_publisher` (终端 1)

```
source ~/ros2_ws/install/setup.bash
export URDF_FILE=$(ros2 pkg prefix my_robot_description)/share/my_robot_description/urdf/diff_dr

ros2 run robot_state_publisher robot_state_publisher \
  --ros-args -p robot_description:="$(cat $URDF_FILE)"
```

成功标志：

```
[INFO] [robot_state_publisher]: got segment base_link
[INFO] [robot_state_publisher]: got segment left_wheel
[INFO] [robot_state_publisher]: got segment right_wheel
```

步骤 2: 启动 joint\_state\_publisher\_gui (终端 2)

```
source ~/ros2_ws/install/setup.bash
ros2 run joint_state_publisher_gui joint_state_publisher_gui
```

弹出 GUI 窗口, 显示 left\_wheel\_joint 和 right\_wheel\_joint 滑块。

步骤 3: 启动 RViz 2 (终端 3)

```
source ~/ros2_ws/install/setup.bash
ros2 run rviz2 rviz2
```

配置 RViz 2:

| 步骤 | 操作                                                        |
|----|-----------------------------------------------------------|
| 1  | Global Options → Fixed Frame → base_link                  |
| 2  | Add → RobotModel → Description Topic: /robot_description  |
| 3  | Add → TF → Show Names: ✓, Show Axes: ✓, Marker Scale: 0.2 |
| 4  | Add → Grid → Cell Size: 0.1                               |

步骤 4: 交互验证

| 操作                      | 预期结果                                     |
|-------------------------|------------------------------------------|
| 拖动 left_wheel_joint 滑块  | 左轮旋转, TF 中 left_wheel 坐标轴变化              |
| 拖动 right_wheel_joint 滑块 | 右轮旋转                                     |
| 查看 TF 面板                | 展开 base_link 看到 left_wheel 和 right_wheel |
| 取消勾选 left_wheel         | 左轮坐标轴消失                                  |

步骤 5: 命令行交叉验证 (终端 4)

```
source ~/ros2_ws/install/setup.bash

# 查看 TF 树
ros2 run tf2_tools view_frames
ls -la frames_*.pdf frames_*.gv

# 查看变换值
ros2 run tf2_ros tf2_echo base_link left_wheel

# 查看话题频率
ros2 topic hz /tf
ros2 topic hz /joint_states
```

## 步骤 6: 保存 RViz 配置

```
File → Save Config As → ~/ros2_ws/src/my_robot_description/rviz/display.rviz
```

更新 `CMakeLists.txt` 安装 rviz 目录:

```
install(DIRECTORY urdf rviz
        DESTINATION share/${PROJECT_NAME}
)
```

后续加载:

```
ros2 run rviz2 rviz2 -d $(ros2 pkg prefix my_robot_description)/share/my_robot_description/rviz/
```

## 创建一键启动脚本

创建 `~/ros2_ws/src/my_robot_description/scripts/display_robot.sh` :



```
#!/bin/bash
# 完整 URDF 可视化启动脚本

set -e

# 环境检查
source /opt/ros/jazzy/setup.bash
source ~/ros2_ws/install/setup.bash

URDF_FILE=$(ros2 pkg prefix my_robot_description)/share/my_robot_description/urdf/diff_drive.urdf
RVIZ_CONFIG=$(ros2 pkg prefix my_robot_description)/share/my_robot_description/rviz/display.rviz

echo "=== URDF 文件: $URDF_FILE ==="
check_urdf $URDF_FILE

echo "=== 启动 robot_state_publisher ==="
ros2 run robot_state_publisher robot_state_publisher \
  --ros-args -p robot_description:="$(cat $URDF_FILE)" &
RSP_PID=$!

sleep 2

echo "=== 启动 joint_state_publisher_gui ==="
ros2 run joint_state_publisher_gui joint_state_publisher_gui &
JSG_PID=$!

sleep 1

echo "=== 启动 RViz 2 ==="
if [ -f "$RVIZ_CONFIG" ]; then
  ros2 run rviz2 rviz2 -d $RVIZ_CONFIG
else
  ros2 run rviz2 rviz2
fi

# 清理
kill $RSP_PID $JSG_PID 2>/dev/null
```

```
chmod +x ~/ros2_ws/src/my_robot_description/scripts/display_robot.sh
```

## 使用系统自带模型快速体验

若尚未创建自己的 URDF，可用 TurtleBot3 快速体验完整流程：

```
sudo apt install -y ros-jazzy-turtlebot3-description ros-jazzy-turtlebot3-fake-node
source /opt/ros/jazzy/setup.bash
export TURTLEBOT3_MODEL=burger

# 一条命令启动完整流程
ros2 launch turtlebot3_fake_node turtlebot3_fake_node.launch.py
```

故障排查清单

| 症状      | 检查步骤                                                                             |
|---------|----------------------------------------------------------------------------------|
| RViz 空白 | <code>ros2 param get /robot_state_publisher robot_description \   head -3</code> |
| 轮子不转    | <code>ros2 topic echo /joint_states --once</code> 检查数据                           |
| TF 报错   | <code>ros2 run tf2_tools view_frames</code> 检查树完整性                               |
| 模型位置不对  | 对比 URDF origin 与 <code>tf2_echo</code> 输出                                        |
| GUI 不弹出 | <code>sudo apt install ros-jazzy-joint-state-publisher-gui</code>                |

关键点

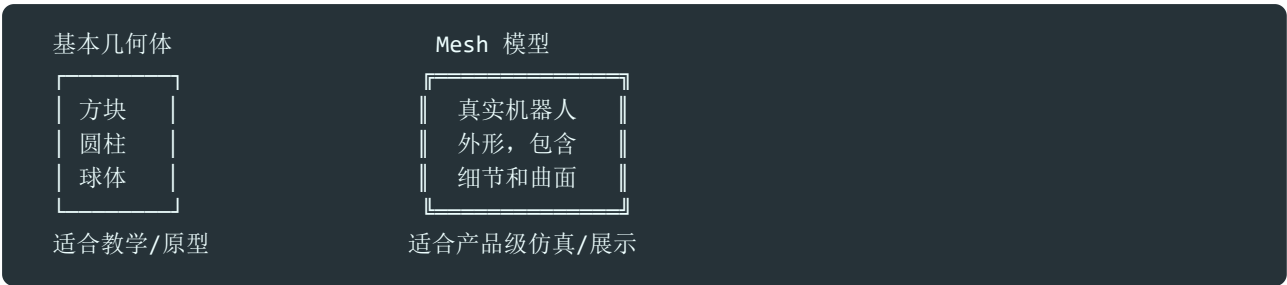
- 完整可视化流程需要三个节点：robot\_state\_publisher + joint\_state\_publisher\_gui + rviz2
  - 启动顺序：先 robot\_state\_publisher，再 joint\_state\_publisher，最后 rviz2
  - 保存 RViz 配置文件 (.rviz) 可避免每次手动添加 Display
  - 命令行验证 (tf2\_echo、view\_frames) 与 RViz 可视化互补
  - 可将启动流程封装为 shell 脚本，实现一键复现
-

# 讲次 10：真实 Mesh 模型快速概览

## 知识点

### 为什么需要 Mesh 模型

基本几何体 (box、cylinder、sphere) 适合简单形状，但真实机器人外形复杂，需要使用 **3D Mesh 模型**。



### 支持的 Mesh 格式

| 格式      | 扩展名  | 特点      | 推荐场景               |
|---------|------|---------|--------------------|
| COLLADA | .dae | 支持材质、动画 | Gazebo / RViz (首选) |
| STL     | .stl | 仅几何，无颜色 | 3D 打印、简单模型         |
| OBJ     | .obj | 广泛支持    | 通用 3D 模型           |

ROS 2 推荐使用 **DAE (COLLADA)** 格式，因为支持内嵌材质颜色。

### 在 URDF 中引用 Mesh

```
<link name="base_link">
  <visual>
    <geometry>
      <mesh filename="package://my_robot_description/meshes/base_link.dae"
        scale="1.0 1.0 1.0"/>
    </geometry>
    <origin xyz="0 0 0" rpy="0 0 0"/>
  </visual>
</link>
```

package:// 协议：

```
package://<package_name>/<relative_path>
```

这是 ROS 特有的路径协议，自动解析为功能包的 share 目录：

```
package://my_robot_description/meshes/base.dae
→ /home/user/ros2_ws/install/my_robot_description/share/my_robot_description/meshes/base.dae
```

## 文件组织结构

```
my_robot_description/
├── meshes/
│   ├── base_link.dae      # 底盘模型
│   ├── wheel.dae          # 轮子模型（可复用）
│   └── sensor.dae         # 传感器外壳
├── urdf/
│   └── my_robot.urdf
├── CMakeLists.txt
└── package.xml
```

CMakeLists.txt 安装 meshes 目录：

```
install(DIRECTORY urdf meshes
        DESTINATION share/${PROJECT_NAME}
)
```

## 使用 TurtleBot3 的 Mesh 模型

TurtleBot3 是最佳的 Mesh URDF 学习范例：

```
source /opt/ros/jazzy/setup.bash

# 查看 Mesh 引用
grep -n 'mesh' \
    /opt/ros/jazzy/share/turtlebot3_description/urdf/turtlebot3_burger.urdf
```

典型片段：

```
<link name="base_link">
  <visual>
    <origin xyz="-0.032 0 0.0" rpy="0 0 0"/>
    <geometry>
      <mesh filename="package://turtlebot3_description/meshes/bases/burger_base.stl"
            scale="0.001 0.001 0.001"/>
    </geometry>
    <material name="light_black"/>
  </visual>
</link>
```

注意 `scale="0.001 0.001 0.001"` —— STL 模型常以毫米为单位，需缩放到米。

scale 属性

| 情况      | scale 值           | 原因         |
|---------|-------------------|------------|
| 模型单位为米  | 1 1 1             | 无需缩放       |
| 模型单位为毫米 | 0.001 0.001 0.001 | mm → m     |
| 模型单位为厘米 | 0.01 0.01 0.01    | cm → m     |
| 模型过大/过小 | 手动调整              | 目视 RViz 判断 |

混合使用基本几何体与 Mesh

实际项目中常混合使用：

```
<robot name="hybrid_robot">
  <!-- 底盘用 Mesh（复杂外形） -->
  <link name="base_link">
    <visual>
      <geometry>
        <mesh filename="package://my_robot_description/meshes/chassis.dae"/>
      </geometry>
    </visual>
  </link>

  <!-- 轮子用圆柱体（简单且关节计算方便） -->
  <link name="left_wheel">
    <visual>
      <geometry>
        <cylinder radius="0.05" length="0.04"/>
      </geometry>
      <origin xyz="0 0 0" rpy="1.5708 0 0"/>
      <material name="black"/>
    </visual>
  </link>
</robot>
```

这是业界常见做法：复杂部件用 Mesh，简单对称部件用基本几何体。

### 获取 Mesh 模型的途径

| 途径       | 说明                                     |
|----------|----------------------------------------|
| 机器人厂商提供  | TurtleBot3、UR、Franka 等官方 description 包 |
| CAD 软件导出 | SolidWorks / Fusion 360 → STL / DAE    |
| 3D 建模软件  | Blender → DAE (可保留材质)                  |
| 在线模型库    | GrabCAD、Thingiverse (注意单位)             |
| AI 生成    | 快速原型 (质量参差不齐)                          |

### CAD 导出注意事项

SolidWorks / Fusion 360 导出清单:

1. 确认单位为米 (m) 或导出后设 `scale`
2. 导出为 STL 或 DAE
3. 简化过于复杂的网格 (面数 < 50K)
4. 将模型原点放在安装面中心
5. 每个 link 导出为独立文件

### 验证 Mesh 模型

```
# 构建并检查文件是否安装
cd ~/ros2_ws
colcon build --packages-select my_robot_description
source install/setup.bash

# 确认 mesh 文件存在
ls $(ros2 pkg prefix my_robot_description)/share/my_robot_description/meshes/

# 启动可视化
ros2 run robot_state_publisher robot_state_publisher \
  --ros-args -p robot_description:="$(cat $(ros2 pkg prefix my_robot_description)/share/my_robot_description/urdf/robot.urdf.xacro)"
ros2 run rviz2 rviz2
```

若 Mesh 未显示: - 检查 `package://` 路径是否正确 - 检查 mesh 文件是否被 `install()` 到 share 目录 - 检查 `scale` 是否导致模型过小/过大 - RViz 中检查 RobotModel 的 Status 是否报错

## Mesh 的性能考量

| 因素        | 建议                 |
|-----------|--------------------|
| 面数        | 每个 link < 50K 三角面  |
| 文件大小      | 单个 mesh < 10MB     |
| collision | 使用简化几何体，不用高精度 mesh |
| 重复部件      | 同一 mesh 文件复用（如左右轮） |

## 关键点

- Mesh 模型（DAE/STL/OBJ）用于描述复杂外形，通过 `<mesh filename="package://...">` 引用
- `package://` 是 ROS 的功能包路径协议，自动解析到 share 目录
- 注意模型单位：STL 常以毫米为单位，需通过 `scale="0.001 0.001 0.001"` 转换
- 实际项目常混合使用 Mesh（复杂部件）和基本几何体（简单部件）
- collision 建议使用简化几何体而非高精度 Mesh，以提升仿真性能